these histories may be exposed to external LLMs. If chat histories containing sensitive information are inadvertently sent to LLMs, there is a potential for the data to be leaked or misused, leading to severe consequences such as identity theft, financial fraud, and privacy violations.

> **NOTE** This example uses `redact-pii`, a simple JavaScript library for redacting PII from user messages. However, for production environments, it's strongly recommended to use `@google-cloud/dlp` for more comprehensive and robust PII detection and anonymization, especially for non-English data or sensitive applications. Also, consider implementing the anonymization logic in your middleware for optimal performance.

Here's how to modify the existing chat API handler to include the client library:

1. Install the `redact-pii` library. You can do this by running

```
$ npm install redact-pii
```

2. Import and initialize the Redactor class:

```
import  { SyncRedactor } from 'redact-pii';
const redactor = new SyncRedactor();
```

3. Create a function for anonymization. Implement a function that uses the `SyncRedactor` to anonymize user messages.

The following listing gives the full code.

Listing 9.7 Anonymizing prompt Input data

```
import  { SyncRedactor } from 'redact-pii';
const redactor = new SyncRedactor();
function anonymizeText(text) {
  return redactor.redact(text);
}

export async function POST(req) {
  try {
    const body = await req.json();
    const validatedData = promptSchema.parse(body);

    const { userId } = await auth();
    if (!userId) {
      return new Response('Unauthorized', { status: 401 });
    }
    const anonymizedInput =
anonymizeText(validatedData.text);
// rest of the code
}
}
```

Identifies and replaces sensitive information (PII) within the text

The result of the anonymizeText is used as the prompt sent to the LLM.